

Objetivos da aula:

- Compreender o conceito de transação e as propriedades ACID (Atomicity, Consistency, Isolation e Durability);
- Utilizar corretamente os comandos BEGIN, COMMIT e ROLLBACK;
- Diferenciar transações implícitas e explícitas no PostgreSQL;
- Aplicar tratamento de exceções em PL/pgSQL para recuperação parcial de operações;
- Identificar o papel das procedures no controle transacional.

Índice:

- i. Introdução ao Controle de Transações
- ii. Recuperação Parcial de Transações com Tratamento de Exceções

i. Introdução ao Controle de Transações

1.1. Introdução

Em bancos de dados relacionais, as operações de modificação de dados (inserções, atualizações e exclusões) precisam garantir **consistência**, **confiabilidade** e **integridade**, mesmo diante de falhas, erros ou acessos concorrentes.

Para isso, os SGBDs oferecem o mecanismo de **controle de transações**, que permite agrupar múltiplas operações SQL em uma única unidade lógica de trabalho. Essa unidade pode ser **confirmada** (commit) ou **desfeita** (rollback), garantindo que o banco de dados permaneça em um estado consistente.

No PostgreSQL, o controle de transações é um dos pilares do funcionamento do sistema.

1.2. O que é uma transação

Uma transação é um conjunto de uma ou mais operações SQL que:

- são executadas como uma **unidade indivisível**;
- produzem um estado consistente do banco;
- podem ser confirmadas (commit) ou desfeitas (rollback).

Definição conceitual:

Uma transação é uma sequência de operações que deve ser executada completamente ou não executada de forma alguma.

1.3. Propriedades ACID

As transações em bancos de dados relacionais seguem o modelo ACID, composto por quatro propriedades fundamentais:

1.3.1 Atomicidade (Atomicity)

- Todas as operações da transação são executadas **por completo** ou **nenhuma delas é aplicada**;
- Em caso de erro, todas as alterações são desfeitas.

Exemplo conceitual:

- inserir uma pessoa e seus carros;
- se uma das inserções falhar, nenhuma alteração deve permanecer.

1.3.2 Consistência (Consistency)

- A transação leva o banco de dados de um estado consistente para outro;
- Todas as regras, constraints e triggers devem ser respeitadas.

1.3.3 Isolamento (Isolation)

- Transações concorrentes não devem interferir indevidamente umas nas outras;
- O resultado deve ser equivalente a uma execução sequencial.

O isolamento será explorado em detalhes em capítulos posteriores.

1.3.4 Durabilidade (Durability)

- Uma vez confirmada (COMMIT), a transação persiste mesmo em caso de falhas;
- Os dados não são perdidos após a confirmação.

1.4. Comandos básicos de controle de transações

1.4.1 BEGIN (ou START TRANSACTION)

Inicia explicitamente uma transação:

```
BEGIN;
```

A partir desse ponto, todas as operações fazem parte da mesma transação.

1.4.2 COMMIT

Confirma definitivamente as alterações realizadas na transação:

```
COMMIT;
```

Após o COMMIT, as alterações tornam-se visíveis para outras transações.

1.4.3 ROLLBACK

Desfaz todas as alterações realizadas desde o início da transação:

```
ROLLBACK;
```

O banco retorna ao estado anterior ao BEGIN.

1.5. Transações implícitas e explícitas

1.5.1 Transações implícitas

No PostgreSQL:

- cada comando SQL é, por padrão, executado em uma transação implícita;
- se o comando for bem-sucedido, ocorre um COMMIT automático;
- em caso de erro, ocorre um ROLLBACK.

Esse comportamento é adequado para operações simples.

```
INSERT INTO pessoa
VALUES (1, 'Ana Maria', '2000-01-01');
INSERT INTO carro
VALUES (1, 'ABC1D23', 2020, 1);
```

1.5.2 Transações explícitas

Utilizadas quando é necessário controlar várias operações como uma unidade:

BEGIN;	BEGIN;
INSERT INTO pessoa	INSERT INTO pessoa
VALUES (2, 'Ana Maria', '2000-01-01');	VALUES (3, 'Ana Maria', '2000-01-01');
INSERT INTO carro	INSERT INTO carro
VALUES (2, 'ABC1D23', 2020, 2);	VALUES (3, 'ABC1D23', 2020, 3);
COMMIT;	ROLLBACK;

Nesse caso, o controle é totalmente explícito ao chegar no COMMIT as operações do bloco são confirmadas e ao chegar no ROLLBACK as operações do bloco são desfeitas.

1.6. Importância do controle de transações

O controle de transações é essencial para:

- evitar dados inconsistentes;
- garantir integridade referencial;
- lidar com falhas de sistema ou aplicação;
- implementar processos complexos;
- suportar ambientes multiusuário.

Sem transações, operações parciais poderiam comprometer o banco de dados.

1.7. Transações e programação no PostgreSQL

O controle de transações é especialmente relevante quando utilizado em conjunto com:

- procedures armazenadas (podem conter COMMIT e ROLLBACK);
- triggers (executados dentro do contexto da transação);

- aplicações que realizam múltiplas operações dependentes.

Importante:

- funções **não** podem controlar transações;
- procedures podem controlar transações.

Essa distinção reforça conteúdos vistos na Aula 5.

ii. Recuperação Parcial de Transações com Tratamento de Exceções

2.1. Introdução

Neste capítulo, o objetivo é compreender como o PostgreSQL permite **recuperação parcial de operações dentro de uma transação**, utilizando tratamento de exceções em PL/pgSQL.

Serão abordados:

- como o PostgreSQL lida com erros em transações;
- uso de blocos BEGIN ... EXCEPTION ... END para controle de falhas;
- recuperação parcial de forma correta em **procedures**;
- diferenças entre rollback total de rollback parcial implícito.

2.2. ROLLBACK total

Como visto no capítulo anterior, o comando:

```
ROLLBACK;
```

desfaz **todas** as operações realizadas desde o início da transação.

Esse comportamento é adequado em transações simples, mas pode ser indesejado quando:

- parte das operações foi concluída com sucesso;
- apenas uma etapa específica falhou;
- deseja-se preservar resultados válidos.

Para esses casos, o PostgreSQL oferece **tratamento estruturado de exceções**.

2.3. Tratamento de exceções em PL/pgSQL

Em PL/pgSQL, o controle de erros é feito por meio do bloco:

```
BEGIN
    -- comandos SQL
EXCEPTION
    WHEN condição THEN
        -- tratamento do erro
END;
```

Características importantes:

- o PostgreSQL cria **automaticamente um ponto de recuperação implícito** no início do bloco;

- se ocorrer um erro dentro do bloco:
 - as operações do bloco são desfeitas;
 - o fluxo segue para a cláusula EXCEPTION;
- a transação **não é encerrada automaticamente**.

Cenário conceitual

Deseja-se executar duas operações:

1. inserir uma pessoa;
2. inserir um carro associado à pessoa.

Se a segunda operação falhar:

- a pessoa deve ser mantida;
- apenas a inserção do carro deve ser descartada.

Exemplo prático em procedure

```
CREATE OR REPLACE PROCEDURE inserir_pessoa_e_carro(  
    IN p_id_pessoa INTEGER,  
    IN p_nome VARCHAR,  
    IN p_nascimento DATE,  
    IN p_id_carro INTEGER,  
    IN p_placa CHAR(7),  
    IN p_ano INTEGER  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    -- Operação principal  
    INSERT INTO pessoa (id_pessoa, nome, nascimento)  
    VALUES (p_id_pessoa, p_nome, p_nascimento);  
  
    -- Bloco com recuperação parcial  
    BEGIN  
        INSERT INTO carro (id_carro, placa, ano, id_pessoa)  
        VALUES (p_id_carro, p_placa, p_ano, p_id_pessoa);  
    EXCEPTION  
        WHEN OTHERS THEN  
            RAISE NOTICE 'Erro ao inserir carro. Pessoa mantida.';  
    END;  
  
    COMMIT;
```

```
END;
$$;
```

Uso

```
-- Executada com sucesso se as tabelas estiverem vazias
CALL inserir_pessoa_e_carro(1, 'Ana Maria', '1999-10-11', 1, 'ABC1D23', 2000);
-- A placa está repetida e lança a mensagem:
-- NOTICE: Erro ao inserir carro. Pessoa mantida.
CALL inserir_pessoa_e_carro(2, 'Maria Clara', '2000-03-04', 2, 'ABC1D23', 2010);
```

Comportamento:

- erro na inserção do carro
- inserção da pessoa mantida
- transação concluída com sucesso

2.4. Relação entre EXCEPTION e transações

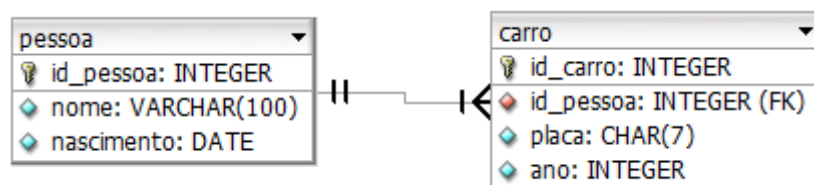
- o bloco EXCEPTION **não encerra a transação**;
- ele apenas desfaz as operações do bloco atual;
- COMMIT e ROLLBACK só podem ser usados em **procedures**, não em funções.

2.5. Boas práticas

- utilizar blocos EXCEPTION para falhas previsíveis;
- manter blocos pequenos e bem definidos;
- evitar capturar erros genéricos (WHEN OTHERS) sem necessidade;
- registrar erros relevantes com RAISE NOTICE ou RAISE EXCEPTION;
- documentar claramente o comportamento esperado.

Exercícios

Para a correta execução dos exercícios, utilize os comandos a seguir para criar as tabelas.



Não precisa carregar os arquivos CSVs.

```
DROP TABLE IF EXISTS carro, pessoa CASCADE;
CREATE TABLE IF NOT EXISTS pessoa (
    id_pessoa INTEGER PRIMARY KEY,
```

```
        nome VARCHAR(100) NOT NULL,  
        nascimento DATE  
    );  
  
CREATE TABLE IF NOT EXISTS carro (  
    id_carro INTEGER PRIMARY KEY,  
    placa CHAR(7) NOT NULL UNIQUE,  
    ano INTEGER,  
    id_pessoa INTEGER NOT NULL,  
    FOREIGN KEY (id_pessoa)  
        REFERENCES pessoa(id_pessoa)  
        ON DELETE CASCADE  
);
```

Exercício 1 — Transação explícita básica

Envolva o comando a seguir em um bloco BEGIN ... COMMIT.

```
INSERT INTO pessoa (id_pessoa, nome, nascimento)  
VALUES (1, 'Ana Silva', '1995-05-20');
```

Resposta:

```
BEGIN;  
  
INSERT INTO pessoa (id_pessoa, nome, nascimento)  
VALUES (1, 'Ana Silva', '1995-05-20');  
  
COMMIT;
```

Exercício 2 — Uso de ROLLBACK

Envolva o comando a seguir em um bloco BEGIN ... ROLLBACK.

```
INSERT INTO pessoa (id_pessoa, nome, nascimento)  
VALUES (2, 'Carlos Souza', '1990-03-10');
```

Resposta:

```
BEGIN;  
  
INSERT INTO pessoa (id_pessoa, nome, nascimento)  
VALUES (2, 'Carlos Souza', '1990-03-10');  
  
ROLLBACK;
```

Exercício 3 — Erro em transação sem tratamento

Considere a transação a seguir. Explique o que ocorre se a segunda inserção gerar erro.

```
BEGIN;  
  
INSERT INTO pessoa (id_pessoa, nome)  
VALUES (3, 'Maria Lima');  
  
INSERT INTO carro (id_carro, placa, id_pessoa)  
VALUES (1, 'ABC1234', 999);  
  
COMMIT;
```

Resposta:

- ocorre erro na inserção do carro (chave estrangeira);
- a transação entra em estado de erro;
- é necessário executar ROLLBACK;
- nenhuma das inserções é confirmada.

Exercício 4 — Recuperação parcial com EXCEPTION

Crie uma procedure que insira uma pessoa e tente inserir dois carros.

Caso a inserção de um dos carros falhe, a pessoa e o outro carro deve ser mantida.

Exemplo de sequência de chamados comandos:

```
-- 1: Limpa as tabelas, inclusive as procedures  
TRUNCATE carro, pessoa CASCADE;  
  
-- 2: Crie a procedure inserir_pessoa_e_dois_carros  
  
-- 3: Deverá inserir 1 pessoa e 2 carros  
CALL inserir_pessoa_e_dois_carros(  
    1, 'Carlos Silva', '1985-10-20', -- Pessoa  
    11, 'AAA1A23', 2020,             -- Carro 1  
    12, 'BBB1B23', 2022             -- Carro 2  
);  
  
-- 4: Verifique as tabelas, repita esses comandos após cada chamada de  
CALL  
SELECT * FROM pessoa;  
SELECT * FROM carro;
```



```
-- 5: Deverá inserir 1 pessoa e o 1o carro
CALL inserir_pessoa_e_dois_carros(
    2, 'Amanda Souza', '1995-11-09', -- Pessoa
    21, 'CCC1C34', 2008,                -- Carro 1
    22, 'AAA1A23', 2018                -- Carro 2 (Placa repetida)
);

-- 6: Deverá inserir 1 pessoa e o 2o carro
CALL inserir_pessoa_e_dois_carros(
    3, 'Maria Clara', '1995-11-09', -- Pessoa
    31, 'AAA1A23', 2014,                -- Carro 1 (Placa repetida)
    32, 'DDD1D23', 2015                -- Carro 2
);
```

Resposta:

```
CREATE OR REPLACE PROCEDURE inserir_pessoa_e_dois_carros(
    IN p_id_pessoa INTEGER,
    IN p_nome VARCHAR,
    IN p_nascimento DATE,
    -- Dados do Carro 1
    IN p_id_carro1 INTEGER,
    IN p_placa1 CHAR(7),
    IN p_ano1 INTEGER,
    -- Dados do Carro 2
    IN p_id_carro2 INTEGER,
    IN p_placa2 CHAR(7),
    IN p_ano2 INTEGER
)
LANGUAGE plpgsql
AS $$
BEGIN
    -- 1. Insere a Pessoa (Se falhar aqui, a procedure inteira para)
    INSERT INTO pessoa (id_pessoa, nome, nascimento)
    VALUES (p_id_pessoa, p_nome, p_nascimento);

    -- 2. Sub-bloco isolado para o Carro 1
    BEGIN
        INSERT INTO carro (id_carro, placa, ano, id_pessoa)
        VALUES (p_id_carro1, p_placa1, p_ano1, p_id_pessoa);
    EXCEPTION
```

```
        WHEN OTHERS THEN
            RAISE NOTICE 'Erro ao inserir Carro 1 (Placa: %). Operação
ignorada.', p_placa1;
        END;

-- 3. Sub-bloco isolado para o Carro 2
BEGIN
    INSERT INTO carro (id_carro, placa, ano, id_pessoa)
    VALUES (p_id_carro2, p_placa2, p_ano2, p_id_pessoa);
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Erro ao inserir Carro 2 (Placa: %). Operação
ignorada.', p_placa2;
    END;

-- O COMMIT final consolidará o que teve sucesso
COMMIT;

END;

$;$
```